Arquitetura da solução

Tecnologias

Camada	Tecnologia	Por quê
Front-end	HTML5 (templates Jinja2), CSS3 puro, JavaScript puro	Leve, sem etapa de build e fácil de manter
Back-end	Python 3.11 + Flask 3	Framework web simples e muito usado no mercado
Persistência	SQLAlchemy 2 (ORM) + Flask-SQLAlchemy	Consultas parametrizadas, independência do SGBD
Banco	SQLite (dev/testes) · PostgreSQL (produção)	SGBDs populares e livres
Testes back-end	pytest + pytest-cov	Padrão de fato em Python
Testes front-end	`node:test` (nativo do Node.js)	Sem dependências extras

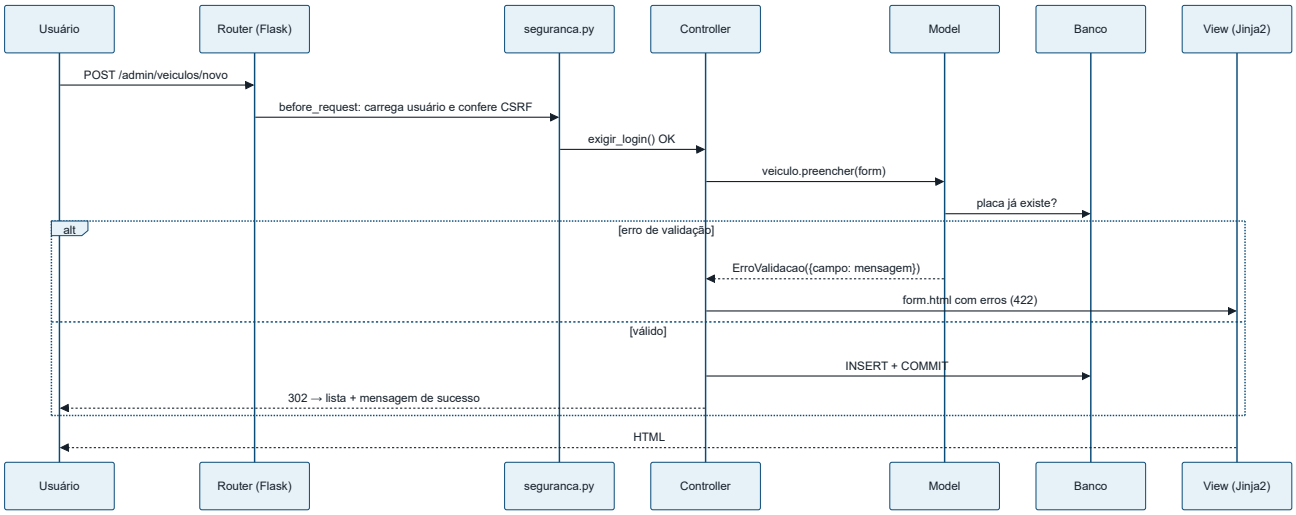
Padrão MVC

```
app/
├── models/          ← MODEL: entidades, validações e regras de negócio
│   ├── veiculo.py       Veiculo.preencher(), buscar_catalogo(), excluir()
│   ├── proposta.py      Proposta.criar(), aceitar(), recusar()
│   ├── venda.py         Venda.registrar()
│   ├── cliente.py       Cliente.validar_dados(), obter_ou_criar()
│   ├── marca.py, usuario.py
│   ├── erros.py         ErroValidacao / ErroRegraNegocio
│   └── util.py          conversão de moeda/número, validação de CPF
├── controllers/     ← CONTROLLER: recebe a requisição, chama o Model, escolhe a View
│   ├── catalogo_controller.py   vitrine pública e envio de proposta
│   ├── auth_controller.py       login/logout
│   ├── painel_controller.py     indicadores
│   ├── veiculo_controller.py    CRUD de veículos + upload
│   ├── marca_controller.py, proposta_controller.py, venda_controller.py
│   └── erro_controller.py       páginas 400/403/404/405/413/500
├── views/           ← VIEW: o que o usuário vê
│   ├── templates/       HTML (Jinja2) por módulo + _macros.html (componentes)
│   ├── static/css/      estilo.css
│   ├── static/js/       validacao.js (regras puras) + app.js (comportamento da tela)
│   └── filtros.py       formatação pt-BR (R$, milhar, datas)
├── servicos/
│   ├── armazenamento.py     gravação e validação das fotos
│   └── limites.py           freio de tentativas repetidas de login
├── seguranca.py     CSRF, sessão, login obrigatório, perfil ADMIN, cabeçalhos HTTP
├── cli.py           comandos `seed` e `criar-usuario`
├── config.py        configurações (produção e testes)
└── __init__.py      create_app(): monta a aplicação (application factory)
```

Responsabilidades

- **Model**: concentra **todas** as regras de negócio. Um Controller nunca muda `veiculo.status` diretamente; ele chama `proposta.aceitar()` ou `Venda.registrar()`. Por isso as regras são testadas sem precisar de HTTP (`tests/test_unitario_models.py`).
- **Controller**: fino. Lê `request.form`, chama o Model, faz `commit` / `rollback`, define o código HTTP (200, 302, 403, 404, 422) e escolhe o template.
- **View**: só apresenta. Recebe dados prontos e formata com filtros (`|moeda`, `|milhar`, `|data_hora`).

Fluxo de uma requisição



Tratamento de erros

Camada	Estratégia
Navegador	`validacao.js` impede o envio e mostra a mensagem no campo; botão desabilitado após enviar (evita duplicidade).
Model	`ErroValidacao` junta **todos** os erros de uma vez (o usuário corrige tudo numa tentativa); `ErroRegraNegocio` para operações proibidas. O objeto só é alterado se tudo for válido.
Controller	`try/except` com `db.session.rollback()`; HTTP 422 para validação, redirecionamento com mensagem para regra de negócio. `IntegrityError` tratado tanto no cadastro de veículo (corrida de placa duplicada, com remoção da foto já gravada) quanto no registro de venda (corrida de duplo envio, barrada por `uq_venda_veiculo`).
Banco	`CHECK`, `UNIQUE`, `NOT NULL` e `FOREIGN KEY` como última barreira.
Aplicação	Páginas amigáveis para 400, 403, 404, 405, 413 e 500; o 500 faz rollback e registra log.

Segurança

Ameaça	Proteção
Senha vazada do banco	Hash com salt (`werkzeug.security`)
CSRF	Token por sessão em todo POST (`seguranca.verificar_csrf`)
XSS	Autoescape do Jinja2 (teste `test_campos_sao_escapados_contra_xss`)
SQL Injection	Só consultas via ORM, parametrizadas
Upload malicioso	Lista de extensões + verificação dos *magic bytes* + nome UUID + limite de 5 MB
Open redirect no login	`destino_seguro()` só aceita caminhos internos
Clickjacking / MIME sniffing	`X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`
Acesso indevido	`exigir_login` nos blueprints do painel e `@somente_admin` nas ações destrutivas
Cookie de sessão	`HttpOnly`, `SameSite=Lax`, `Secure` em produção e validade de 8 horas
Chave de sessão fraca	Em produção a aplicação **recusa iniciar** com a chave de desenvolvimento (`app/__init__.py`)

Força bruta no login	5 falhas em 10 minutos bloqueiam por 15 (`servicos/limites.py`), resposta HTTP 429
Alteração de cadastro alheio	O formulário público reutiliza o cliente pelo e-mail, mas não reescreve nome e telefone de quem já existe (RN13)

Ambiente e variáveis

Variável	Para que serve	Padrão
`AUTOVENDA_AMBIENTE`	`producao` liga as proteções de produção (cookie `Secure`, exigência de chave própria)	`desenvolvimento`
`SECRET_KEY`	Chave de assinatura da sessão. **Obrigatória** quando o ambiente é `producao`	chave de desenvolvimento
`DATABASE_URL`	Conexão do SGBD; sem ela, usa SQLite em `instance/`	SQLite local

Gere uma chave com `python -c "import secrets; print(secrets.token_hex(32))"`.

Limitação conhecida do freio de login: a contagem fica na memória do processo, o que basta para a loja rodar em um processo só. Em implantação com vários processos ou máquinas, a contagem precisa ir para armazenamento compartilhado (Redis ou uma tabela no banco). A classe `ControleTentativas` foi escrita com essa troca em mente — só a implementação muda, a interface não.

Desempenho das listagens

As telas do painel mostram, em cada linha, dados de tabelas relacionadas (cliente, veículo, marca, vendedor). Sem cuidado, isso vira uma consulta por linha — o problema conhecido como **N+1**. As quatro listagens usam carregamento antecipado (`joinedload` nas relações e `contains_eager` onde o `join` já existe por causa do filtro), e há teste automatizado que conta as consultas e falha se o número voltar a crescer junto com o número de linhas (`test_d01_*`).

Manual de uso — AutoVenda

Parte 1 — Instalação

Requisitos

- Python 3.11 ou superior
- (Opcional) Node.js 18+ para rodar os testes do front-end
- (Opcional) PostgreSQL 14+ para produção

Passo a passo (Windows, PowerShell)

```
git clone https://github.com/Gerson201/autovenda.git
cd autovenda
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
flask --app run seed        # cria usuários, marcas e 9 veículos de exemplo
flask --app run run         # abre em http://127.0.0.1:5000
```

Passo a passo (Linux/macOS)

```
git clone https://github.com/Gerson201/autovenda.git
cd autovenda
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
flask --app run seed
flask --app run run
```

Usuários de demonstração (criados pelo `seed`)

Perfil	E-mail	Senha
Administrador	admin@autovenda.com	Admin@123
Vendedor	vendedor@autovenda.com	Vende@123

> Troque essas senhas antes de publicar o sistema. Para criar outro usuário: `flask --app run criar-usuario`.

Variáveis de ambiente (produção)

Variável	Exemplo	Uso
`SECRET_KEY`	uma sequência longa e aleatória	Assina o cookie de sessão. **Obrigatório trocar.**
`DATABASE_URL`	`postgresql+psycopg://user:senha@host/autovenda`	Banco de produção (instale `psycopg[binary]`)

Parte 2 — Visitante (cliente)

Encontrar um carro

1. Acesse a página inicial (**Catálogo**).
2. Digite marca ou modelo na busca (ex.: *Corolla*) e clique em **Buscar**.
3. Use os **Filtros** à esquerda (no celular, acima da lista): marca, preço mínimo/máximo, ano, combustível, câmbio e ordenação. Clique em **Aplicar filtros**.
4. Para recomeçar, clique em **Limpar filtros**.

Carros com o selo **Reservado** estão em negociação com outro cliente, mas ainda aceitam propostas.

Enviar uma proposta

1. Clique no cartão do carro para abrir os detalhes.
2. No quadro **Fazer uma proposta**, preencha nome, e-mail, telefone com DDD e o valor oferecido. A mensagem é opcional.
3. Clique em **Enviar proposta**. Se algum campo estiver errado, a mensagem aparece em vermelho logo abaixo dele.
4. Ao enviar, aparece a confirmação *"Proposta enviada com sucesso!"*. Um vendedor entrará em contato.

Parte 3 — Vendedor e administrador

Entrar e sair

- Clique em **Área do vendedor**, informe e-mail e senha e clique em **Entrar**.
- Para sair, clique em **Sair (seu nome)** no menu.

Painel

Mostra quantos veículos estão disponíveis, reservados e vendidos, as propostas pendentes e o faturamento. Clique em um indicador para abrir a lista correspondente.

Cadastrar um veículo

1. **Veículos › + Cadastrar veículo**.
2. Preencha:

- **Placa:** `ABC1234` ou Mercosul `ABC1D23` (pode digitar com hífen ou minúsculas).
- **Ano do modelo:** igual ao de fabricação ou o ano seguinte.
- **Preço:** só números, com ponto ou vírgula nos centavos.
- **Foto:** JPG, PNG ou WEBP de até 5 MB. A prévia aparece assim que você escolhe o arquivo.

1. Clique em **Cadastrar veículo**. O carro aparece no catálogo na hora.

Editar ou excluir

- Na lista, clique em **Editar**. Para trocar a foto, basta escolher outra.
- **Excluir** aparece só para administradores e pede confirmação. As propostas do veículo também são apagadas.

- Veículos **vendidos** não podem ser editados nem excluídos, porque fazem parte do histórico.

Marcas (administrador)

Marcas › digite o nome › **Adicionar**. Só é possível excluir marcas sem veículos.

Tratar propostas

1. **Propostas** abre na aba **Pendentes**.
2. **Aceitar**: o veículo fica **Reservado** e sai da negociação com outros clientes. Só pode haver uma proposta aceita por carro.
3. **Recusar**: a proposta é encerrada. Se era a aceita, o carro volta a **Disponível**.
4. Use o e-mail do cliente (link) ou o telefone para entrar em contato.

Registrar uma venda

- **A partir da proposta** (recomendado): na aba **Aceitas**, clique em **Registrar venda**. Os dados do cliente e o valor já vêm preenchidos. Complete o **CPF**, ajuste o valor final se necessário, escolha a **forma de pagamento** e confirme.
- **Venda direta**: **Vendas › + Registrar venda** ou **Vender** na lista de veículos.

Depois da venda, o carro sai do catálogo, as outras propostas dele são recusadas automaticamente e o valor entra no faturamento.

Parte 4 — Problemas comuns

Situação	Solução
"Sua sessão expirou. Recarregue a página…"	A página ficou aberta muito tempo ou o servidor reiniciou. Recarregue (F5) e envie de novo.
"Cadastre ao menos uma marca…"	Cadastre a marca em **Marcas** (perfil administrador).
"Arquivo muito grande"	Reduza a foto para menos de 5 MB.
Botão **Excluir** não aparece	Sua conta é de vendedor; só administradores excluem.
Página 404 em um veículo	O carro foi vendido e saiu do catálogo.

Parte 5 — Rodar os testes

```
pytest --cov=app                    # back-end (unitários, controllers, integração)
node --test "tests/frontend/*.test.js"   # front-end
```